Beyond RDBMS: Transparent Observation via Multi-Dimensional Key-Value Layers in AI-Native Architectures

Author: Koji Yokoyama (unizou)
Repository: https://github.com/unizou/microforce-coer

Abstract

Traditional Relational Database Management Systems (RDBMS) have long relied on static schemas, explicit relational mappings, and rigid migration processes. However, in the era of autonomous AI agents, these legacy constraints introduce unnecessary friction. This paper introduces "Microforce," a novel architectural paradigm that eliminates predefined relations and structural migrations entirely. By leveraging a multi-dimensional Key-Value Store (KVS), Microforce treats data, schemas, and execution logic as independent, transparent layers. Instead of utilizing procedural business logic (e.g., explicit iterations and conditional branching), the system employs "Transparent Observation." In this process, AI agents project an intent (a target key) across these layers, allowing the optimal data state to converge and crystallize instantly without explicit structural joins. We demonstrate that this non-von Neumann approach significantly reduces

architectural overhead, and we provide a production-ready blueprint available as an open-source repository.

1. Introduction

For decades, software engineering has been anchored to the von Neumann architecture and relational data models. Developers spend significant time defining database schemas, managing Object-Relational Mappers (ORMs), and writing explicit business logic to join, filter, and manipulate data. While this approach was optimal for traditional computing, it creates a bottleneck for LLMs (Large Language Models) and AI agents that thrive on dynamic, intent-driven operations. This paper challenges the fundamental necessity of static relations. We propose that by treating a Key-Value Store (KVS) not merely as a temporary cache, but as a multi-dimensional semantic space, we can completely bypass traditional database constraints.

2. The Microforce Paradigm

2.1 Elimination of Migrations and Relations

In the Microforce architecture, data is stored as flat semantic layers in a KVS. There are no foreign keys or relational tables. When a new data structure or feature is required, a new independent "layer" is simply added to the KVS. Because layers do not strictly depend on each other at rest, structural migrations become obsolete.

2.2 Multi-Dimensional Layering

Microforce visualizes data as a "mille-feuille" (layered pastry). For any given entity, its raw data, validation schemas, UI representation, and state exist on separate, transparent planes sharing the same semantic key.

3. Transparent Observation

The core innovation of Microforce is the execution mechanism defined as "Transparent Observation." Traditional programming relies on explicit control flow (`if/else`, `for/while`) to merge data and apply logic. In contrast, Transparent Observation aligns the multi-dimensional layers along a targeted vector (the Intent Key). An execution engine (the "Architect") observes through these layers simultaneously. The intersection of the requested schema layer and the raw data layer automatically crystallizes the result. Logic is no longer executed procedurally step-by-step; rather, it resolves instantly based on the observation of aligned semantic layers.

4. Implementation Details

To validate the theoretical framework, we developed the `microforce-coer` library, utilizing Python and SQLite/PostgreSQL as the underlying KVS engines. The architecture comprises two core entities:

1. **`MicroforceLayer`**: Represents a single semantic surface storing JSON payloads or vector embeddings.
2. **`MicroforceArchitect`**: The engine that aligns keys across multiple `MicroforceLayer` instances, performing the transparent observation to return a strongly-typed materialized object without SQL joins.

This separation ensures that AI agents interact only with the `Architect`, passing intent via semantic keys, while the underlying storage mechanism effortlessly absorbs any schema variance. All source code, including demonstration payloads, is available under the MIT License at our official repository: https://github.com/unizou/microforce-coer.

5. Conclusion

The Microforce architecture fundamentally redefines the boundary between data storage and application logic. By utilizing a multi-dimensional semantic KVS and transparent observation, we remove the friction of ORMs, RDBMS migrations, and procedural coding paradigms. As software development transitions into an AI-driven era, architectures must evolve from rigid, explicitly programmed pipelines to fluid, intent-crystallizing spaces. Microforce serves as the foundational blueprint for this transition.